COMPRENDRE LE PROJET de A à Z

Project Resilience-Alpha — documentation pédagogique de l'équipe

Tout ce qui a été construit, comment ça marche, et pourquoi

À lire par tous les membres avant la soutenance

Équipe Workshop2026-M2-G01 · EPSI/myDiL · Octobre 2026

0. À quoi sert ce document

Ce document explique **l'intégralité** du projet Resilience-Alpha, pièce par pièce, dans un langage simple avec des analogies. Objectif : que **chaque membre de l'équipe** puisse comprendre et ré-expliquer ce qui a été fait, même les parties qu'il n'a pas codées. Il se lit du début à la fin, mais chaque section est autonome.

Deux autres fichiers l'accompagnent : le **Dossier** (document officiel d'ingénierie et de gouvernance, plus formel) et le **Guide oral** (ce qu'il faut dire sur chaque slide le jour J).

1. Le projet en une page

Une société financière fictive exploite une **plateforme de paiement**. Quand elle tombe en panne ou subit une attaque, aujourd'hui un humain doit intervenir : c'est lent, manuel, non reproductible. Notre mission : démontrer qu'on peut rendre cette plateforme **cyber-résiliente**, c'est-à-dire capable de **détecter seule un incident, d'y réagir et de prouver qu'elle est rétablie**, sans intervention humaine.

On a donc construit un vrai petit système de paiement (fictif), on l'a mis sous surveillance par de l'intelligence artificielle, on lui a ajouté un « robot réparateur », et on l'attaque pour montrer que toute la chaîne fonctionne.

La chaîne, en trois temps (à retenir absolument)

- **Détecter** : un modèle de machine learning surveille les métriques et repère l'anomalie.
- **Décider & agir** : un moteur de remédiation choisit l'action corrective et l'exécute.
- **Vérifier** : la plateforme prouve que le métier remarche (une vraie transaction fictive aboutit).

Analogie : c'est comme une voiture qui détecte elle-même une crevaison, gonfle la roue de secours, et fait un tour de parking pour vérifier qu'elle roule à nouveau — sans que le conducteur descende.

2. L'architecture d'ensemble

La plateforme est faite de **petits services indépendants** (microservices), chacun dans son **conteneur** Docker. Un conteneur, c'est une « boîte » qui emballe un programme avec tout ce dont il a besoin pour tourner à l'identique sur n'importe quelle machine. Les conteneurs sont orchestrés par **Docker Compose** (un chef d'orchestre qui les démarre tous ensemble).

Règle d'or de sécurité : **un seul service est exposé à l'extérieur, le WAF**. Tous les autres (base, API, secrets, supervision) restent sur un réseau privé interne. On réduit ainsi la « surface d'attaque » au minimum.

Il y a **13 services** au total. Les sections suivantes expliquent chacun d'eux.

3. Les composants, un par un

Composant	Rôle	Ce qu'il fait	Techno
WAF (nginx)	Reverse-proxy / pare-feu applicatif	C'est le portier unique. Toutes les requêtes externes passent par lui. Il filtre les attaques évidentes (injections SQL, scripts), limite le débit (anti-déni de service) et cache les services internes. C'est le seul composant publié sur le réseau.	nginx

Composant	Rôle	Ce qu'il fait	Techno
API paiement	Le cœur métier (FastAPI, Python)	Elle crée les comptes et exécute les transactions : débiter un compte, créditer un autre, de façon atomique (tout ou rien) grâce aux transactions de la base. Elle expose aussi ses métriques (débit, latence, erreurs) pour la surveillance.	FastAPI
Auth (JWT)	Service d'authentification	Il délivre des jetons signés (JWT) à courte durée de vie. L'API vérifie ce jeton avant d'exécuter une transaction. Analogie : un bracelet de festival qu'on scanne à chaque entrée.	PyJWT
PostgreSQL	La base de données	Elle stocke comptes et transactions. Ses transactions ACID garantissent qu'un paiement n'est jamais à moitié fait. Elle n'est JAMAIS exposée : seule l'API peut lui parler.	Postgres 16
Vitrine	Interface de paiement (React/Tailwind)	La page web où l'on clique « Payer le marchand ». Elle sert à montrer le métier en action avant/après l'incident.	React
Détection ML	Le radar à anomalies (scikit-learn)	Toutes les 3 secondes, il lit 4 métriques depuis Prometheus et demande à un modèle de machine learning si le comportement est normal. Si c'est anormal, il prévient la remédiation. Détaillé en §4.	scikit-learn
Remédiation	Le robot réparateur	Il reçoit l'incident, choisit une action (redémarrer un service, isoler un conteneur…), l'exécute, puis VÉRIFIE que le métier remarche. Détaillé en §5.	Docker SDK
Assistant RAG	L'aide à l'opérateur (Ollama + LangChain)	Un chatbot local qui répond aux questions de l'opérateur en cherchant dans les procédures. En lecture seule, sans aucun pouvoir d'action. Détaillé en §6.	Ollama
Dashboard	La tour de contrôle (React/Tailwind)	La page qui montre en temps réel l'état des services, la détection, les événements de remédiation et l'assistant. C'est ce que le jury regarde pendant la démo.	React
Prometheus / Grafana / Loki	L'observabilité	Prometheus collecte les chiffres (métriques), Grafana les met en graphiques, Loki centralise les journaux (logs). Cette chaîne alimente réellement la détection ML.	stack Grafana

4. La détection d'anomalies (le cerveau)

Le problème. Comment savoir qu'une plateforme « va mal » sans écrire des centaines de règles du type « si la latence dépasse X alors alerte » ? Ces règles sont fragiles et le sujet les interdit. On utilise donc un vrai modèle de machine learning.

L'idée. On apprend au modèle à quoi ressemble une « journée normale » de la plateforme, décrite par 4 indicateurs. Le modèle trace une « enveloppe » autour de la zone normale. Toute mesure qui en sort nettement est signalée comme anomalie.

Indicateur	Signification	Un écart signifie…
req_rate	requêtes par seconde	débit anormal → possible déni de service
error_ratio	part de réponses en erreur (5xx)	erreurs anormales → injection ou panne
latency_p95	temps de réponse (95e pct)	lenteur anormale → saturation / base
inflight	requêtes en cours	embouteillage → service en difficulté

Le modèle, en clair

On utilise un **détecteur de nouveauté à covariance robuste** (EllipticEnvelope de scikit-learn). Il calcule la « distance de Mahalanobis » : à quel point un point est loin du nuage normal, en tenant compte de la façon dont les indicateurs varient ensemble. Point clé : le **seuil de décision n'est pas écrit à la main**, il est calibré

automatiquement sur les données normales (on tolère ~0,5 % de fausses alertes).

Anecdote utile pour le jury : on a d'abord essayé un « Isolation Forest », mais il ratait certains écarts extrêmes portés par une seule variable. On a changé pour la covariance robuste, plus fiable, et on l'a validé par des tests automatiques (cas normal + 3 types d'anomalies, tous corrects).

Pour chaque alerte, le service indique le **driver principal** (quel indicateur a le plus dérapé) et une phrase d'explication, puis notifie le moteur de remédiation.

5. La remédiation autonome (les bras)

C'est **le cœur de la notation**. Le moteur ferme la boucle en 5 temps :

```
Détection → Décision → Action → Vérification → Clôture (ou nouvelle tentative, max 3)
```

La politique de décision

Selon l'indicateur en cause, le moteur applique un plan prédéfini :

Anomalie (driver)	Action décidée	Pourquoi
latency_p95 / base injoignable	redémarrer la base puis l'API	lever la contention
req_rate (déni de service)	redémarrer l'API	vider la file, absorber le pic
error_ratio	redémarrer l'API + ouvrir une investigation	rétablir ET enquêter
inflight	redémarrer l'API	désengorger

Reprise RÉELLE, pas cosmétique

Un conteneur qui redémarre ne prouve rien. Le moteur ne déclare la reprise que si **la vérification métier réussit** : la base est lisible ET une **nouvelle transaction fictive aboutit**. C'est la différence entre « le voyant est repassé au vert » et « le service rend vraiment service ».

Le garde-fou du composant privilégié

Le moteur a des droits élevés (il pilote les conteneurs). Pour qu'il ne devienne pas lui-même une porte d'entrée, il **n'expose aucun shell distant** : il n'accepte qu'une **liste fermée d'actions** prédéfinies, toutes journalisées, avec un jeton à portée limitée. Et il ne peut pas être piloté par l'assistant RAG.

6. L'assistant opérateur (RAG)

Deuxième brique d'IA : un assistant qui aide l'humain à **retrouver la bonne info** dans les procédures pendant un incident. Il est construit avec **Ollama** (un modèle de langage qui tourne en local, < 8 milliards de paramètres) et **LangChain** (l'orchestration).

RAG = *Retrieval-Augmented Generation*. En clair : au lieu d'inventer, l'assistant **va d'abord chercher** les passages pertinents dans nos documents, puis rédige une réponse à partir d'eux, en citant ses sources. Analogie : un étudiant qui ouvre d'abord son cours avant de répondre, au lieu de répondre de mémoire.

Garde-fou essentiel : l'assistant est en **lecture seule**. Il aide à comprendre, il n'agit jamais sur le système. Il ne se confond pas avec le moteur de remédiation.

7. La sécurité (défense en profondeur)

La sécurité n'est pas une couche unique mais **plusieurs barrières** :

- **WAF** : filtre les injections et limite le débit en entrée.
- **Durcissement** : conteneurs sans droits root, capacités réduites, option no-new-privileges, surface minimale.
- **SIEM** : journaux centralisés dans Loki/Grafana, les requêtes bloquées sont corrélables pour l'investigation.

- **Secrets** : gérés avec SOPS (chiffrés) ou Vault — aucun mot de passe en clair dans le dépôt.

8. Les deux scénarios d'attaque et les résultats

Depuis un poste « attaquant », strictement en labo, on provoque deux incidents :

Scénario A — déni de service volumétrique (via le WAF)

On inonde le WAF de requêtes. Résultat mesuré sur 30 s : **~10 820 requêtes, dont 10 486 bloquées (429) par le WAF, soit 97 %**. Le backend n'a presque pas été touché. **Message : la défense périmétrique fonctionne.**

Scénario B — attaque qui franchit le périmètre (directe sur l'API)

On frappe l'API directement (en simulant un attaquant qui a contourné le WAF). Cette fois rien ne limite le débit : **req_rate explose → la détection ML lève une anomalie → la remédiation redémarre le service et vérifie la reprise**. **Message : quand une attaque passe, l'IA prend le relais.**

Ensemble, ces deux histoires montrent une **défense en profondeur** : chaque couche joue son rôle.

Indicateur	Valeur	Signification
RTO	< 60 s	délai de reprise automatique, sans humain
RPO	~ 0	aucune perte de données (base transactionnelle)
Flood bloqué	97 %	efficacité du WAF sur le scénario A

9. La gouvernance (justifier les choix)

La résilience n'est pas que technique. Trois livrables de gouvernance accompagnent la solution (détaillés dans le Dossier) :

- **EBIOS Risk Manager** : analyse structurée des risques (valeurs métier, sources de menace, scénarios comme le DoS, l'injection, la fuite de secret, l'abus du moteur) et leur traitement.
- **PCA/PRA** : plan de continuité et de reprise adapté à la solution (RTO < 60 s, RPO ~0), distinguant bien restaurer la disponibilité et corriger une compromission.
- **NIS2 / DORA** : ce qu'un prototype de 4 jours peut démontrer (gestion du risque, détection, tests de résilience) vs ce qu'il ne peut qu'esquisser (notification réglementaire d'incident).

10. Lancer et démontrer la plateforme

Démarrage

```
docker compose up -d --build
docker compose ps # vérifier que les 13 services sont Up
```

Points d'accès

Quoi	Adresse
Dashboard de supervision	http://localhost:3001
Vitrine de paiement	http://localhost:8090
Grafana (graphiques)	http://localhost:3000 — admin / admin-demo

Dérouler la démo du jury

- **État sain** : payer depuis la vitrine → transaction « settled » ; dashboard tout vert.

- **Scénario A** : `python redteam\dos_controlled.py --duree 30 --rps 120 --jaccepte-labo` → le WAF bloque ~97 %.
- **Scénario B** : `python redteam\dos_controlled.py --target http://localhost:18000 --rps 150 --duree 15 --jaccepte-labo` → anomalie détectée, remédiation, reprise.
- **Preuve de reprise** : re-payer depuis la vitrine → la nouvelle transaction aboutit.

Le Clip Chrono (vidéo ≤ 60 s) doit capturer, sans coupe : l'attaque → l'anomalie → la réaction → une nouvelle transaction réussie.

11. Carte du dépôt (où est quoi)

Dossier	Contenu
payment-si/	le SI de paiement : api (FastAPI), auth (JWT), db (SQL), frontend (vitrine)
ml-detection/	le détecteur d'anomalies (modèle + boucle de surveillance)
remediation/	le moteur de reprise (politique + actions Docker)
rag-assistant/	l'assistant opérateur (RAG) + le corpus de procédures
dashboard/	la supervision React/Tailwind
infra/	waf, prometheus, grafana, loki, promtail, ansible
security/	durcissement + gestion des secrets (SOPS/Vault)
redteam/	scripts d'attaque de labo (bornés, défensifs)
docs/	EBIOS, PCA/PRA, NIS2/DORA, scénario de démo
tests/	tests automatisés (détection, schémas, RAG)
docker-compose.yml	l'orchestration de toute la plateforme

12. Questions probables du jury (et réponses)

Q. Pourquoi pas de simples règles « si X alors alerte » ?

R. Elles sont fragiles et interdites par le sujet. Le modèle ML apprend la normalité et s'adapte, sans seuils écrits à la main.

Q. Un conteneur qui redémarre, est-ce vraiment une reprise ?

R. Non, et c'est pour ça qu'on VÉRIFIE : santé profonde + une nouvelle transaction fictive qui aboutit. Sinon on relance (max 3 fois).

Q. Le moteur de remédiation n'est-il pas un risque de sécurité ?

R. Il a des droits élevés, donc on le bride : liste fermée d'actions, pas de shell distant, journalisation, jeton limité.

Q. Pourquoi le WAF a bloqué l'attaque mais la détection n'a rien vu (scénario A) ?

R. Parce que le WAF a trop bien filtré : le backend n'a pas été stressé. C'est voulu. Le scénario B montre l'autre cas, où l'attaque passe et l'IA réagit.

Q. Est-ce prêt pour la production ?

R. Non, et on l'assume : prototype de 4 jours. Limites documentées (SOPS vs Vault, pas de mTLS interne, PC Hôte unique). Ce qu'on revendique : une chaîne crédible, exécutée, testée, défendable.

Q. Les données sont-elles réelles ?

R. Non, tout est fictif. Aucune donnée bancaire réelle n'est utilisée.

Project Resilience-Alpha — Comprendre le projet · Données et contexte entièrement fictifs · Équipe Workshop2026-M2-G01 · EPSI/myDiL, Octobre 2026.